Rapport de performance

Site d'Ina Zaoui : pages du Front Office · Symfony 8.1 / PHP 8.4 / PostgreSQL 16 · 11 novembre 2026

101 → 1	**−47 %**	**−41 %**	**6,1 Mo → 44 Ko**
requêtes SQL sur « Invités »	temps Symfony « Invités »	temps de réponse prod « Invités »	image de l'accueil

1. Objectif et méthode

La note de cadrage signale que la page « Invités » ralentit à mesure que le nombre d'invités augmente. L'objectif est d'en **identifier la cause**, de la **corriger**, puis de mesurer le temps d'affichage de **toutes les pages du Front Office** avant et après correction.

Outils retenus

- **Symfony Web Profiler** (environnement `dev`) : intégré au framework, gratuit, il donne pour chaque requête le temps d'exécution, la mémoire, le détail des requêtes SQL et le nombre d'entités Doctrine chargées. C'est l'outil le plus adapté pour trouver la cause d'une lenteur côté serveur. Les profils ont été relevés automatiquement par un script (lecture du jeton `X-Debug-Token` puis des collecteurs `time`, `db` et `memory`).
- **Chronométrage en environnement `prod`** (`APP_DEBUG=0`, cache préchauffé), pour mesurer le temps de réponse réel sans le surcoût du profiler.
- **API Navigation/Resource Timing du navigateur** (Chrome) pour le chargement complet côté client et le poids des images.

New Relic ou Blackfire auraient demandé un agent à installer et un compte ; pour un site de cette taille, le profiler Symfony suffit et peut être réutilisé par Ingrid sans coût.

Protocole

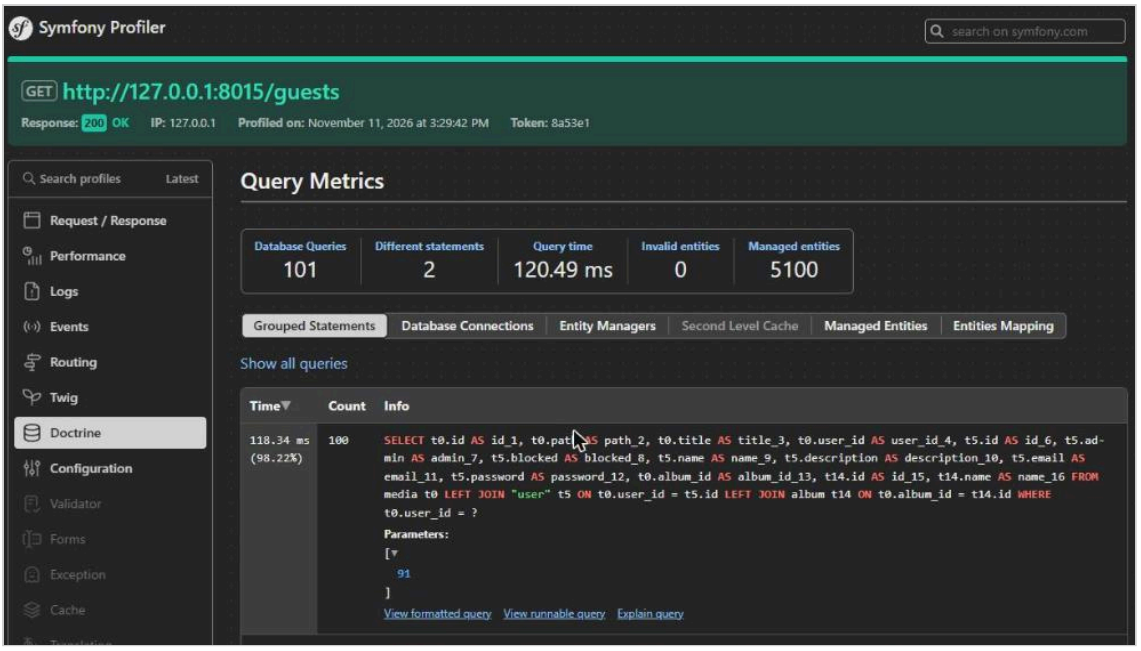
- Données réelles : dump du backup (101 utilisateurs, 5 050 médias, 5 albums) et images d'origine.
- Pages : `/`, `/guests`, `/guest/{id}` (invité avec 50 photos), `/portfolio`, `/portfolio/{album}`, `/about`.
- 1 requête de chauffe, puis 10 mesures par page (dev) et 20 mesures par page (prod). On retient la **médiane**.
- Poste local Windows, serveur PHP intégré, PostgreSQL 16 sous Docker. Les valeurs absolues dépendent de la machine ; ce sont les **écarts avant/après** qui comptent.

Indicateurs suivis

Indicateur	Pourquoi
Temps d'exécution Symfony (ms)	temps passé par l'application pour produire la page
Nombre de requêtes SQL	révèle les problèmes N+1 et la charge sur la base
Mémoire PHP (Mo)	grossit avec le nombre d'entités hydratées
Temps de réponse prod (ms)	ce que perçoit réellement le visiteur côté serveur
Chargement complet et poids des images	temps d'affichage côté navigateur

2. Diagnostic de la page « Invités »

Le profiler montre **101 requêtes SQL** pour afficher la liste : 1 requête pour les invités, puis **1 requête par invité** (100 fois la même requête sur `media`). Doctrine charge aussi **5 100 entités** en mémoire alors que seuls des noms et des compteurs sont affichés.

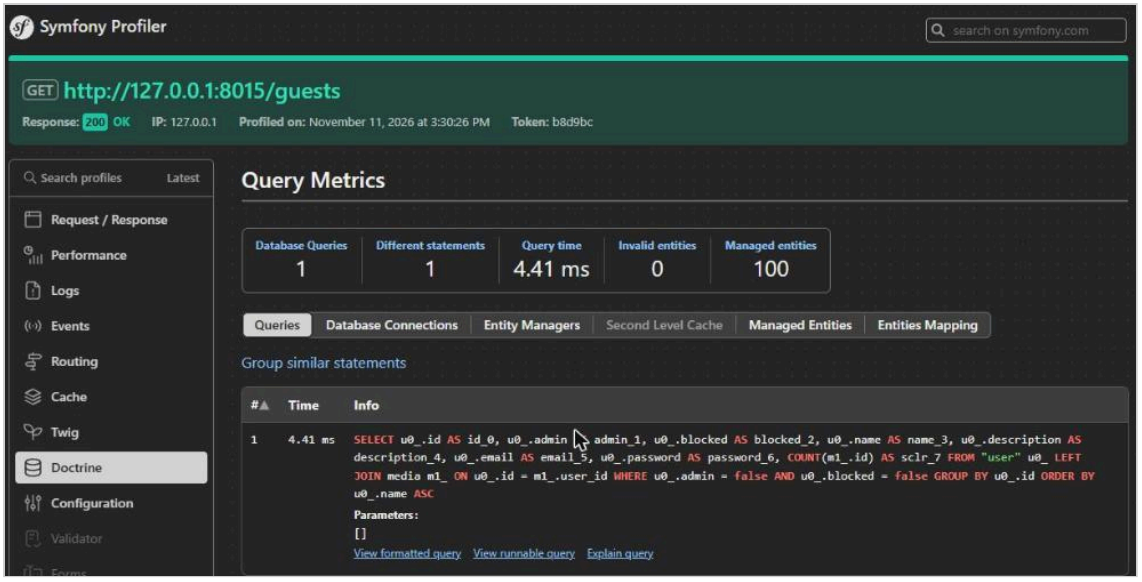


> **Cause :** le template affichait `{{ guest.medias|length }}`. Pour compter les photos, Twig déclenche le chargement complet de la collection `medias` de chaque invité (relation paresseuse), avec en plus des jointures inutiles sur l'utilisateur et l'album dues au `fetch: EAGER` de l'entité `Media`. C'est le problème classique du **N+1** : le coût augmente avec le nombre d'invités et le nombre de photos.

3. Correctifs appliqués

1. **Une seule requête agrégée** : `UserRepository::findGuestsWithMediaCount()` fait un `LEFT JOIN media` + `COUNT(m.id)` + `GROUP BY u.id`, filtré sur les invités non bloqués. Le template affiche directement le compteur.
2. **Suppression du `fetch: EAGER`** sur `Media::user` et `Media::album`. La liste d'administration charge l'artiste et l'album dans la même requête (`findPageForAdmin`), donc pas de N+1 non plus.
3. **Images du Front** en `loading="lazy"` : seules les photos visibles sont téléchargées.

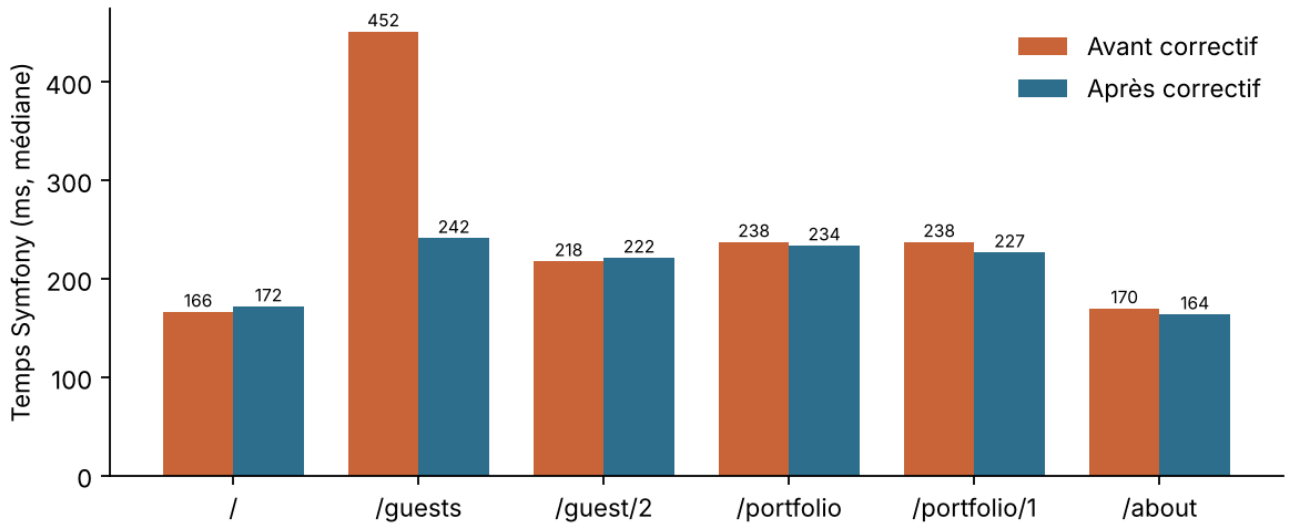
4. **Image d'accueil** redimensionnée : 4096×2803 px (6,1 Mo) → 1600 px de large (44 Ko).
5. **Test de non-régression** : un test fonctionnel vérifie que `/guests` n'exécute qu'**une** requête SQL.



Après correctif : 1 requête, 4,4 ms de temps SQL (au lieu de 120 ms), 100 entités gérées (au lieu de 5 100).

4. Résultats par page (environnement dev, Symfony Profiler)

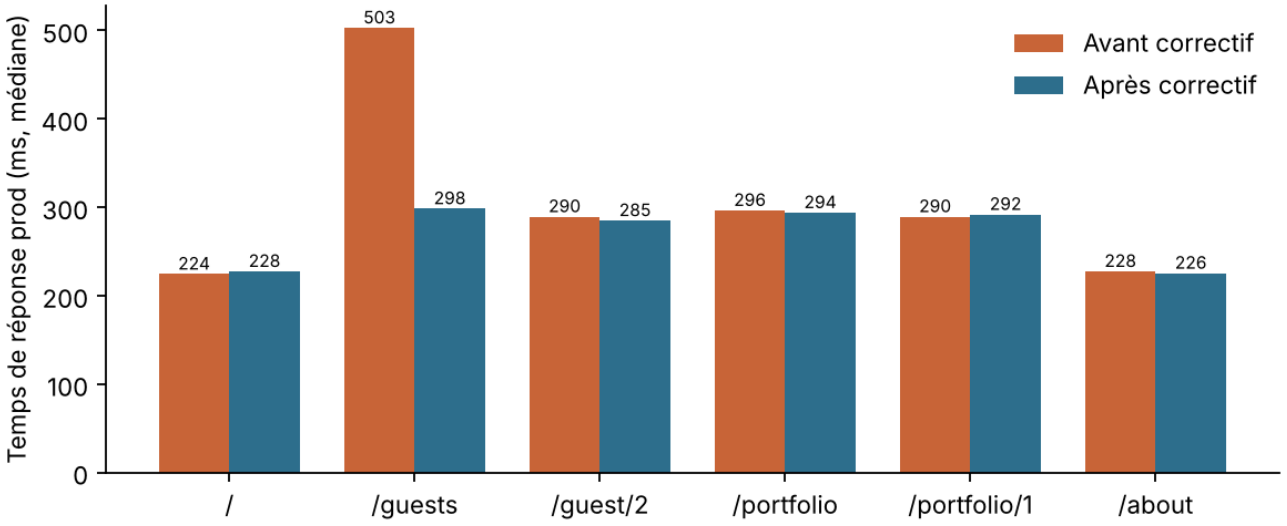
Page	Temps Symfony avant	après	Requêtes SQL avant	après	Mémoire avant	après
Accueil `/`	167 ms	172 ms	0	0	28 Mo	28 Mo
Invités `/guests`	**452 ms**	**242 ms**	**101**	**1**	**34 Mo**	**28 Mo**
Invité `/guest/{id}`	218 ms	222 ms	2	2	30 Mo	28 Mo
Portfolio `/portfolio`	238 ms	234 ms	3	3	30 Mo	28 Mo
Album `/portfolio/{id}`	238 ms	227 ms	3	3	30 Mo	28 Mo
Qui suis-je `/about`	170 ms	164 ms	0	0	30 Mo	30 Mo



Point de référence : sur le site d'origine (Symfony 5.4), la page Invités faisait déjà 101 requêtes (395 ms). La migration n'a pas causé la lenteur : elle était présente dès l'origine.

5. Temps de réponse en production (APP_ENV=prod)

Page	Avant (médiane)	Après (médiane)	Gain
Accueil	225 ms	228 ms	≈
Invités	**503 ms**	**299 ms**	**−41 %**
Invité	290 ms	285 ms	≈
Portfolio	297 ms	294 ms	≈
Album	290 ms	292 ms	≈
Qui suis-je	228 ms	226 ms	≈



Le socle d'environ 200 ms par requête est propre au serveur PHP intégré sous Windows ; il est identique avant et après, donc l'écart mesuré vient bien du correctif.

6. Côté navigateur (Chrome, après correctifs)

Page	Chargement complet	Images téléchargées	Poids images	Poids sans lazy-loading
Accueil	~500 ms → **~220 ms**	3 / 3	6 137 Ko → **48 Ko**	-
Invités	~275 ms	1 / 1	< 1 Ko	-
Invité (50 photos)	~275 ms	13 / 51	2 794 Ko	**11 706 Ko**
Portfolio (50 photos)	~285 ms	13 / 51	2 829 Ko	**10 474 Ko**
Album	~280 ms	11 / 11	2 287 Ko	2 287 Ko
Qui suis-je	~215 ms	2 / 2	201 Ko	201 Ko

Avec le lazy-loading, une page de 50 photos ne télécharge à l'ouverture que les photos visibles : environ 2,8 Mo au lieu de 11,7 Mo, soit **−76 %**.

7. Conclusion et recommandations

- La lenteur de la page « Invités » venait d'un **N+1 Doctrine** déclenché depuis Twig. Elle est corrigée : **1 requête au lieu de 101**, temps divisé par près de 2, et un coût désormais **constant** quel que soit le nombre de photos par invité.
- Les autres pages du Front font 0 à 3 requêtes. Leur temps serveur est stable et sain.
- Pistes pour la suite (Ingrid) :
    - générer des **miniatures** à l'upload (les photos font ~220 Ko chacune, affichées en petit), par exemple avec LiipImagineBundle ;
    - **paginer** la page invité et le portfolio au-delà de 50 photos ;
    - activer l'**OPcache preload** et un **cache HTTP** (Cache-Control) sur les pages publiques ;
    - ajouter un index sur `user(admin, blocked)` si le nombre d'invités devient très grand.

Les données brutes (CSV) et le script de mesure sont dans `docs/performance/` du dépôt.